

Database Architecture of World-Class Marketplaces

How hyperscale commerce platforms — Amazon, Alibaba/Taobao, Shopee, Coupang — structure their data stores to withstand hundreds of thousands of concurrent users, mapped directly onto the schema this project already ships.

- Polyglot Persistence
- Database-per-Service
- Catalog / Offer Model
- Sharding
- CQRS
- SAGA

01 Why a Single Database Cannot Scale

Putting everything into one large PostgreSQL/MySQL works only up to modest scale. At 100k concurrent users the failure points are well defined.

100k+ Concurrent users (target)	90% read Read : write imbalance	Single PK Write contention point	N patterns Distinct access shapes
---	---	--	---

- **Write contention** — stock decrement and order creation crowd onto single rows. A single master's connections and locks become the ceiling.
- **Read explosion** — product and search reads are 10–100× writes. Serving that from a transactional DB starves the writes too.
- **Divergent access patterns** — consistency-critical payments, flexible product documents, millisecond session/cart, and full-text search each demand a *fundamentally different* store.
- **Fault isolation** — the search index can die while payments must stay alive. One database is one failure domain.

The solution has two axes. **Horizontal (Polyglot Persistence)** — use a different *kind* of database per data character. **Vertical (Sharding + Replication)** — split each database across nodes and add read replicas.

02 Polyglot Persistence — One Store per Job

Amazon's principle of "one service = one database, the right kind of database for that service." This project's stack is exactly this picture.

■ PostgreSQL 16

WRITE / OLTP

The consistency-critical **source of truth**: orders, payments, inventory, settlements. ACID transactions, constraints and foreign keys guarantee integrity.

This project · orders , payments , inventories ,
agent_settlements , saga_states

■ MongoDB 7

READ / QUERY

The CQRS **read model**. Denormalized documents — product detail, order history — read in one shot without joins. Flexible schema absorbs per-product variance.

This project · product/order query views populated by sync-worker from Kafka

■ Redis 7 Cluster

CACHE / LOCK

Millisecond, high-frequency, volatile data: sessions, cart (Hash), cache-aside, and the **distributed lock** that prevents overselling of stock.

This project · cart, sessions, Lua-scripted stock-reservation lock

■ OpenSearch 2

SEARCH / FACET

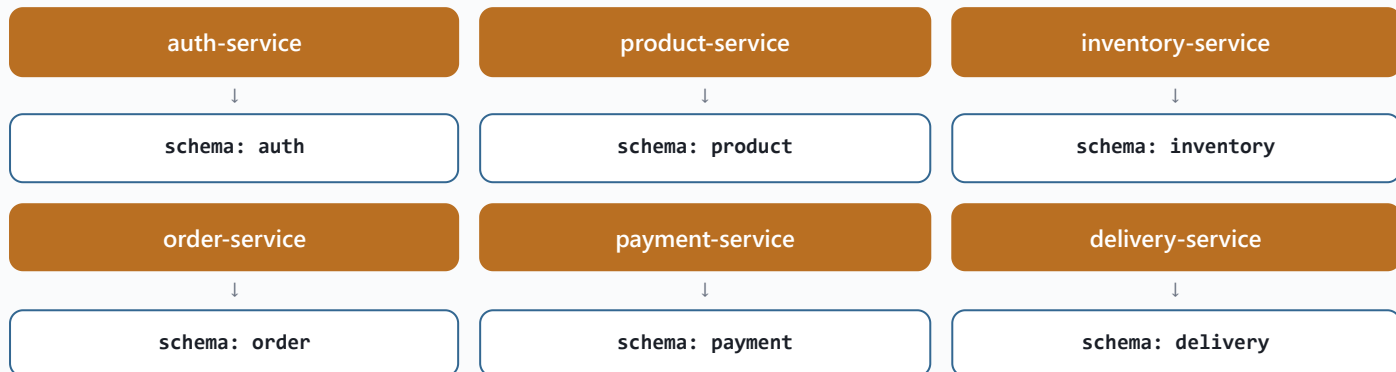
Inverted-index **full-text search, autocomplete and faceted filters** with Korean morphology (nori). Quality and speed a RDB `LIKE '%...%'` can never reach.

This project · product search index, cache-aside with Redis

At scale, platforms add **Kafka** (replay the event log like a database), **Cassandra/DynamoDB** (ultra-high-write logs, carts, sessions), and **ClickHouse/OLAP** (analytics). Here, Kafka is the backbone of CQRS synchronization.

03 Database-per-Service — Isolated Schemas

The MSA rule: a service never reads another service's tables directly. This project enforces it with `schema` boundaries inside one PostgreSQL — a pragmatic compromise under the offline constraint.

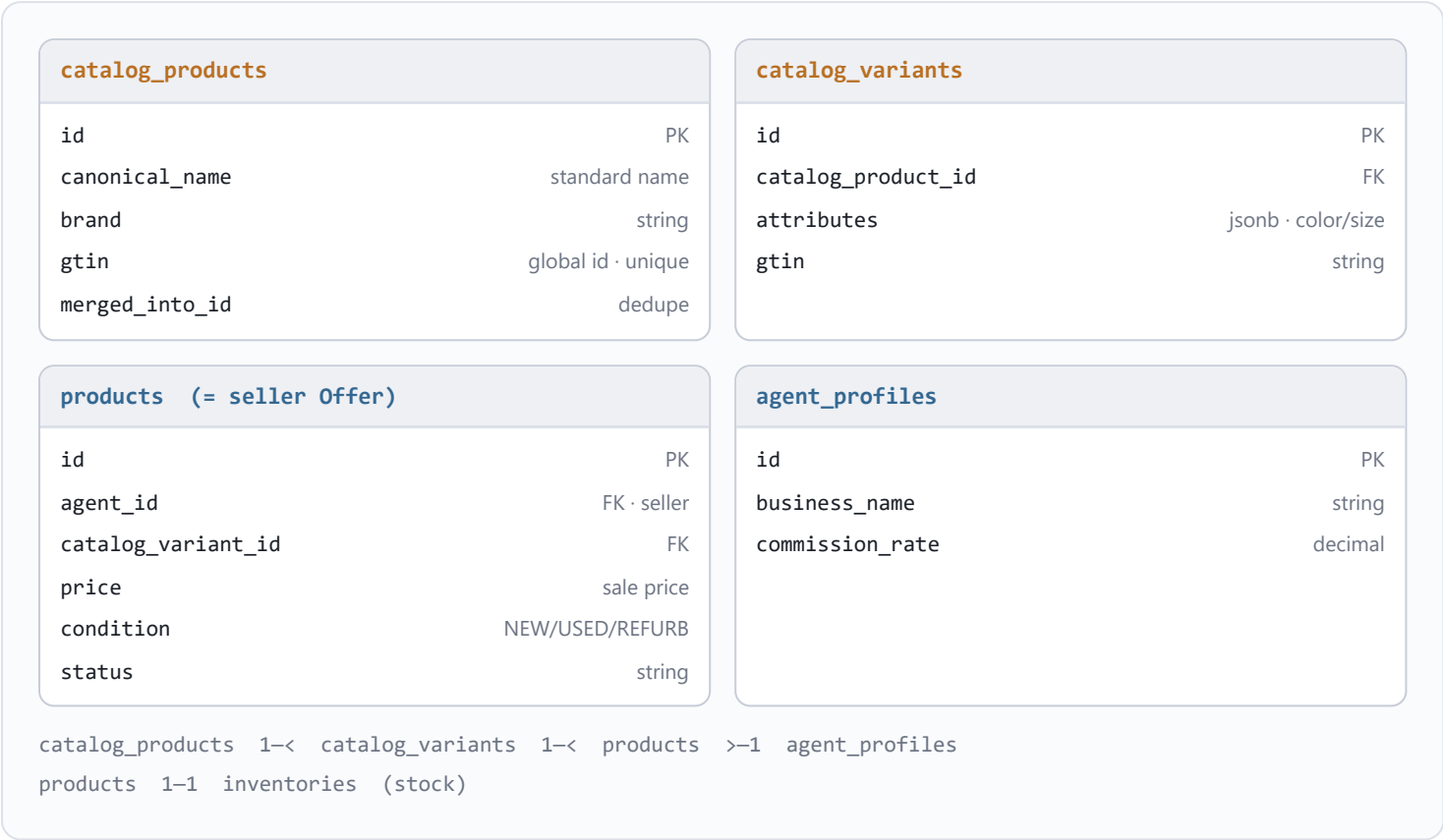


Cross-boundary references are **logical (ID only)**, not physical `FOREIGN KEY`s — marked in the SQL as `-- ref: auth.users.id`.

Why no physical FK across services? To break tight coupling. If an auth user is deleted, the order service must not break at the DB level. Consistency is reconciled by **events (SAGA) and application code** — the Amazon/Uber microservice data-ownership principle.

04 Catalog / Offer — the Heart of a Marketplace

How do you model "many sellers sell the same product"? Amazon's **ASIN (catalog)** ↔ **Offer (seller)** split is the answer, and this project's `14_marketplace_catalog.sql` implements exactly that.



Three key design decisions (from the actual schema)

- GTIN uniqueness

`catalog_products.gtin` is globally unique — prevents the same physical product being registered twice.
- Offer = products

One `products` row is an **offer**: "seller A sells this variant at this price / this condition." SKU is unique per `(agent_id, sku)`, not per catalog.
- No duplicate offer

`uq_products_active_offer (agent_id, catalog_variant_id, condition)` — a seller cannot post two active offers for the same variant & condition.

This split is what enables **"12 sellers offer this product, sorted by lowest price"** (Buy Box), catalog dedupe (`merged_into_id`), and attaching reviews to the catalog rather than the offer — the defining marketplace features.

05 Sharding — Which Key to Split On

When a table reaches billions of rows it no longer fits on one server. Platforms distribute data across nodes by a **shard key**; the key choice decides success.

DATA	SHARD KEY	RATIONALE
Orders / Cart	<code>user_id</code>	"All my orders" lands on one shard; traffic spreads per user.
Products / Inventory	<code>agent_id</code> or <code>catalog_id</code>	Seller dashboards and stock management get locality.
Payments / Settlement	<code>order_id</code>	Payment is 1:1 with order; co-locate with the order flow.
Event log	time range + hash	Append-only; archive/drop old partitions wholesale.

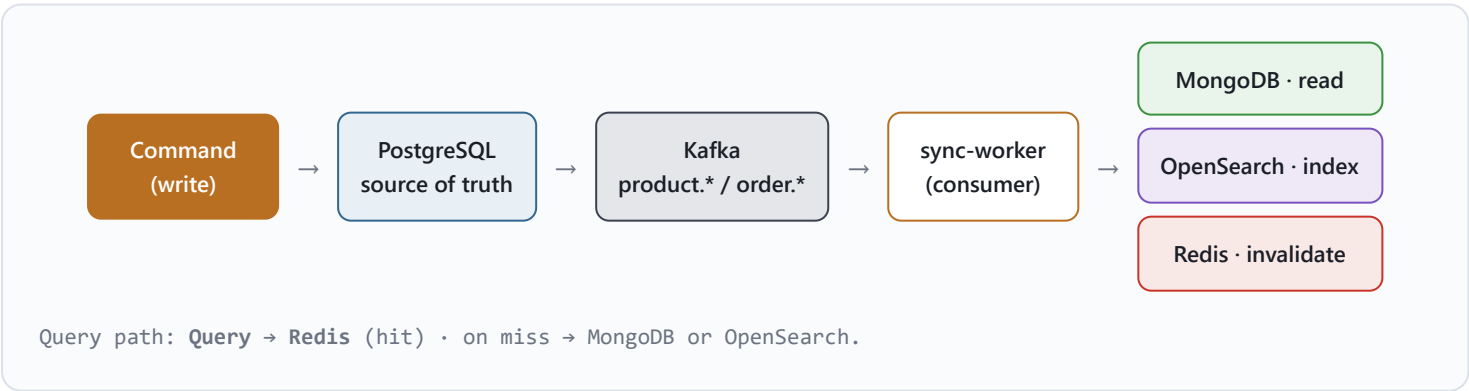
Techniques

- **Consistent hashing** — minimizes keys remapped when nodes join/leave. The default in Redis Cluster and Cassandra.
- **Range vs hash** — range favors scans but risks hotspots; hash spreads evenly but blocks range queries. Usually blended.
- **Read replicas** — master takes writes only; many replicas absorb reads. The physical form of CQRS.
- **Globally-unique IDs** — auto-increment breaks once sharded. Use `UUID` (this project) or Snowflake IDs.

Making every PK a `UUID` here is no accident — it is the choice that lets you scale across shards and services without ID collisions. At offline scale it is still a single node, but the schema is already **sharding-ready**.

06 CQRS — Physically Splitting Read and Write

Writes go to normalized PostgreSQL; reads to denormalized MongoDB/OpenSearch. Kafka events reconcile the two with **eventual consistency**.



Trade-off: there is a lag of tens of ms to a few seconds before a write appears in the read model. UX like "order not visible immediately after payment" must be designed around this lag (optimistic UI, client-side echo of the write response).

07 Core Transactional ERD — Order / Payment / Settlement

The write domain bound by SAGA. An order holds items from multiple sellers, and a payment splits into per-seller **settlements** — a marketplace-specific shape.



- Idempotency keys

Unique on `orders` / `payments` — retries never turn into double orders or double charges. Essential in distributed systems.
- Order snapshot

`order_items` **copies** name and price. Later seller price changes never mutate past orders, and cross-service joins are avoided.
- Per-seller settlement

One payment decomposes into many `agent_settlements`. The platform keeps `commission_rate` and pays the seller `net_amount`.
- SAGA state machine

`saga_states` tracks reserve → pay → ship; on mid-way failure, a **compensating transaction** rolls back.

08 What Each Global Platform Actually Runs

Same principles, different implementations. Representative stores each chose for its scale.

PLATFORM	WRITE / OLTP	READ · DOC · CACHE	SEARCH	SIGNATURE
Amazon	Aurora (MySQL/PG), DynamoDB	DynamoDB (cart/session), ElastiCache	In-house (A9)	Dedicated DB per service; DynamoDB hyper-scale.
Alibaba / Taobao	OceanBase, PolarDB, sharded MySQL (TDDL)	Tair (Redis-like), HBase	In-house	Self-built distributed SQL for Singles' Day scale.
Shopee	Sharded MySQL	Redis, TiDB	Elasticsearch	Multi-region sharding; TiDB for horizontal scale.
Coupang	MySQL/PostgreSQL, Aurora	DynamoDB, Redis	Elasticsearch/OpenSearch	Event-driven MSA + large- scale Kafka.
This project	PostgreSQL 16 (per schema)	MongoDB 7, Redis 7 Cluster	OpenSearch 2 (nori)	Offline-only · Kafka CQRS · SAGA · UUID sharding- ready.

The point is not **what** they run but **why they split**. Every hyperscale platform, without exception, ① separates stores by data character (Polyglot), ② isolates ownership per service (DB-per-Service), ③ separates catalog from offer, and ④ separates write from read (CQRS). This project's schema already embodies all four.